

Design and Implementation of Directed Graph Algorithms in C++

Heidi Tønnesson

March 2026

1 Graph Representation

This project implements two different representations of directed graphs using an abstract class. An incidence list and an adjacency matrix. A graph consists of nodes (vertices) and directed edges connecting them. Multiple edges between the same nodes and self-loops are allowed.

Abstract class

An abstract class **IGraph** defines a common interface for graph operations such as insertion, deletion, and file I/O. The methods are implemented differently in each concrete graph representation, allowing the same interface to be used for both the incidence list and adjacency matrix structures.

Incidence List

The incidence list representation models the graph using explicit **Node** and **Edge** objects. Each node has a unique label and stores its outgoing edges, while each edge stores references to its source and target nodes. Graphs can be read from and written to file in a simple triple format, where each line represents a directed edge.

Adjacency matrix

A second implementation uses an adjacency matrix. Here, nodes are stored as objects, while edges are represented implicitly as lists of labels in a 2D matrix. This supports directed graphs and multiple edges.

Deletion and Rule of five

The incidence list implementation uses dynamic memory allocation (**new** and **delete**) and therefore follows the rule of five to ensure correct resource management. In contrast, the matrix-based implementation relies on standard containers, which handle memory automatically and do not require an explicit implementation of the rule of five.

2 Graph Algorithms

Tarjan's algorithm for SCC

Tarjan's algorithm is used to compute the strongly connected components (SCCs) of a directed graph. An SCC is a set of nodes where every node is reachable from every other node in the set.

The algorithm performs a depth-first search (DFS) and assigns each node an index and a lowlink value. A stack is used to keep track of active nodes during the search. When a node has the same index and lowlink value, it is identified as the root of an SCC, and all nodes belonging to that component are removed from the stack.

The algorithm is implemented as a separate class and operates on the graph through the **IGraph** interface, making it independent of the underlying graph representation.

Diamond-shaped paths

The diamond-shaped path problem consists of finding all pairs of nodes (n_i, n_j) such that there exist two distinct paths from n_i to n_j , following two given sequences of edge labels.

The implementation reads two sequences of labels from a file and, for each node, follows both sequences through the graph. The resulting sets of reachable nodes are compared, and pairs that appear in both sets are reported.

The algorithm uses only the interface provided by **IGraph**, and is therefore independent of the underlying graph representation.

Makefile

A Makefile is provided to compile the program and run demonstration cases. (`scc_demo`, `diamond_demo`, `io_demo`, `delete_demo`).

Runtime Tarjans SCC

The runtime increases clearly with the size of the graph for both representations. The incidence-list implementation is consistently faster than the adjacency-matrix implementation. Since the generated graphs have approximately $E \propto n$, the observed behaviour is consistent with the expected $O(V + E)$ complexity, i.e. it exhibits near-linear scaling in n , with some deviations due to constant factors and memory effects.

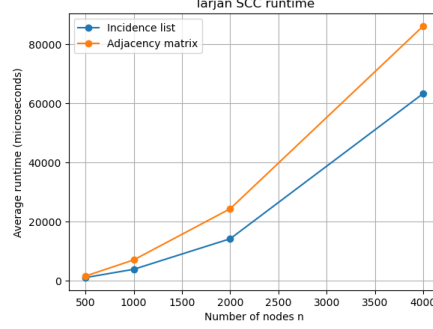


Figure 1: Runtime for Tarjan's SCC as a function of number of nodes n .

Runtime Diamond Query

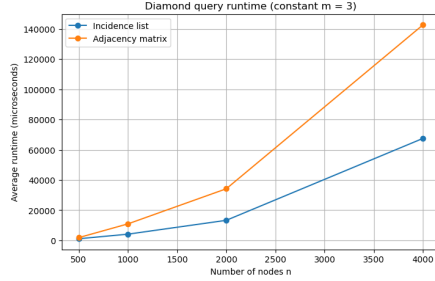


Figure 2: (a) Constant m

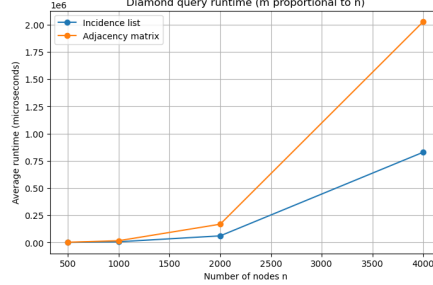


Figure 3: (b) $m \propto n$

For constant query length m , the runtime increases mainly with the graph size n , and the incidence-list implementation is consistently faster. When m grows proportionally with n , the runtime increases much more rapidly, since both the graph and the query become larger.

ER-diagram

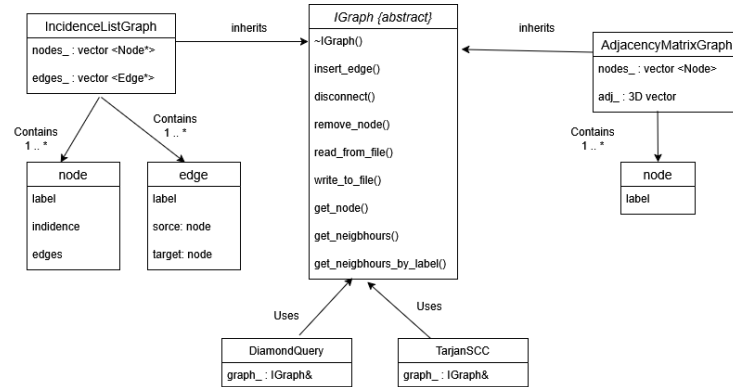


Figure 4: Entity-relationship diagram of the graph/class architecture.

The diagram illustrates the class structure of the system. The abstract interface **IGraph** defines the common operations for graph manipulation and is implemented by both **IncidenceListGraph** and **AdjacencyMatrixGraph**.

In the incidence-list representation, nodes and edges are explicit objects, whereas in the adjacency-matrix representation nodes are stored explicitly and edges are represented implicitly in the matrix.

The classes **TarjanSCC** and **DiamondQuery** operate on the **IGraph** interface, allowing the same algorithms to be applied to both representations without modification.